# Cahier des charges Frontend – PWA Tontine

Nov 2, 2026 · @Akawa

Ce document est destiné à Claude dans VS Code pour construire tout le frontend de l'application de tontine, avec des données simulées en attendant le backend.

## 1. Objet et consignes pour l'agent

Construis le frontend complet d'une PWA de gestion de tontines, en français, pensée d'abord pour le téléphone, avec une couche de données simulée remplaçable plus tard par Supabase.

**Contexte métier.** Une tontine est un groupe de personnes qui cotisent un montant fixe à chaque période (par exemple 4 500 FCFA par mois). À chaque tour, un membre reçoit la cagnotte (cotisation × nombre de membres). Chaque membre reçoit la cagnotte une seule fois par cycle. Le montant, la fréquence et le nombre de membres sont propres à chaque groupe.

**Consignes de travail.**

- Avance étape par étape selon la section 11 ; à la fin de chaque étape, l'application doit démarrer sans erreur.
- Ne connecte aucun vrai backend : toutes les données passent par la couche `services/` décrite en section 8.
- Toute l'interface est en français ; les montants sont en FCFA.
- Respecte strictement la charte graphique de la section 3 ; ne choisis pas d'autres couleurs.
- Écris du TypeScript strict, sans `any`.
- Si une règle de ce document est ambiguë, pose la question avant de coder plutôt que d'inventer.
- Ne génère pas de clés API ni de secrets dans le code front.

## 2. Stack technique et installation

React avec Vite et TypeScript, stylé avec Tailwind CSS, rendu installable avec vite-plugin-pwa.

| Besoin | Bibliothèque |
| --- | --- |
| Base | React 18, Vite, TypeScript (mode strict) |

| Besoin | Bibliothèque |
|---|---|
| Style | Tailwind CSS 3 |
| PWA | vite-plugin-pwa (Workbox) |
| Navigation | React Router 6 |
| Données et cache | TanStack Query |
| Formulaires | React Hook Form + Zod |
| Icônes | lucide-react |
| Dates | date-fns (locale fr) |
| État global léger | Zustand (session et utilisateur connecté) |
| Notifications toast | sonner |
| Tests | Vitest + Testing Library |
| Qualité | ESLint + Prettier |

```
npm create vite@latest tontine-pwa -- --template react-ts
cd tontine-pwa
npm install react-router-dom @tanstack/react-query react-hook-form zod
@hookform/resolvers zustand lucide-react date-fns sonner
npm install -D tailwindcss@3 postcss autoprefixer vite-plugin-pwa vitest
@testing-library/react @testing-library/jest-dom jsdom
npx tailwindcss init -p
```

Variables d'environnement prévues (fichier `.env.example`, valeurs vides) : `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_VAPID_PUBLIC_KEY`, `VITE_USE_MOCKS=true`.

## 3. Charte graphique

Le blanc domine, le bleu porte les actions et la navigation, le vert clair signale l'argent, la réussite et les éléments mis en valeur.

| Rôle | Nom du token | Hex | Usage |
|---|---|---|---|
| Fond principal | `white` | #FFFFFF | Fond des pages et des cartes |
| Fond secondaire | `mist` | #F4F8FB | Fond derrière les cartes, zones neutres |

| Rôle | Nom du token | Hex | Usage |
|---|---|---|---|
| Bleu principal | `primary` | #1E5AA8 | Boutons principaux, liens, barre de navigation active |
| Bleu foncé | `primary-dark` | #12407A | Survol et appui des boutons, titres forts |
| Bleu clair | `primary-light` | #E3EEFA | Fond des puces et badges d'information |
| Vert clair | `mint` | #A8E6C1 | Accents, illustration, carte « cagnotte » |
| Vert très clair | `mint-light` | #E8F8EF | Fond des états « payé » et des encarts positifs |
| Vert succès | `success` | #2E9E5B | Texte et icône « payé », montants reçus |
| Orange alerte | `warning` | #D98A1C | « En attente », date limite proche |
| Rouge erreur | `danger` | #D64545 | « En retard », erreurs de formulaire |
| Texte principal | `ink` | #1B2733 | Texte courant |
| Texte secondaire | `muted` | #5B6B7A | Libellés, aides, dates |
| Bordures | `line` | #DCE5EE | Bordures de cartes et séparateurs |

Déclare ces couleurs dans `tailwind.config.ts` sous `theme.extend.colors` et n'utilise jamais de couleur en dur dans les composants. Le texte sur fond `mint` reste `ink` pour garder un bon contraste ; le texte blanc n'est utilisé que sur `primary` ou `primary-dark`.

**Typographie.** Police Inter (Google Fonts) avec repli `system-ui, sans-serif`. Tailles : titre de page 24 px gras, titre de carte 18 px semi-gras, texte 16 px, libellés 14 px. Les montants s'affichent en chiffres tabulaires ( `tabular-nums` ).

**Formes.** Coins arrondis 12 px pour les cartes, 10 px pour les boutons et champs. Ombres très légères ( `shadow-sm` ). Espacements par pas de 4 px ; marge de page 16 px sur mobile.

**Boutons.** Principal : fond `primary`, texte blanc, hauteur 48 px, pleine largeur sur mobile. Secondaire : fond blanc, bordure et texte `primary`. Action d'argent (Payer) : fond `success`, texte blanc. Destructif : texte `danger`, fond blanc.

**Badges de statut.** Payé = fond `mint-light`, texte `success`. En attente = fond orange très clair, texte `warning`. En retard = fond rouge très clair, texte `danger`. Ton tour = fond `primary-light`, texte `primary`.

**Icône et manifeste PWA.** `theme_color` #1E5AA8, `background_color` #FFFFFF. Icône provisoire : cercle bleu avec trois pièces vert clair, en 192 et 512 px, plus une version maskable.

**Mode sombre.** Hors périmètre pour l'instant ; structure quand même les couleurs en tokens pour pouvoir l'ajouter.

## 4. Rôles et routes

L'application compte plusieurs administrateurs : un super administrateur gère la plateforme et les comptes admin, et chaque groupe peut avoir un ou plusieurs administrateurs.

| Rôle | Portée | Peut faire |
|---|---|---|
| Super administrateur | Toute la plateforme | Créer, suspendre ou supprimer des comptes administrateurs ; voir tous les groupes et les statistiques globales |
| Administrateur | Les groupes dont il est propriétaire ou co-admin | Créer des groupes, les configurer, inviter et gérer les membres, suivre les paiements, déclencher les reversements, ajouter des co-admins à ses groupes |
| Participant | Les groupes dont il est membre | Rejoindre un groupe, payer, consulter le calendrier et l'historique |

Un administrateur est aussi un utilisateur : il peut être participant dans d'autres groupes. Le rôle dans un groupe ( `owner` , `co_admin` , `member` ) est distinct du rôle global ( `super_admin` , `admin` , `user` ). Seul le propriétaire d'un groupe peut le supprimer ou retirer un co-admin.

**Navigation.** Barre d'onglets en bas sur mobile (Accueil, Groupes, Notifications, Profil) ; un onglet Administration apparaît pour les rôles `admin` et `super_admin` . Sur écran large (≥ 1024 px), la barre devient un menu latéral gauche.

| Route | Accès | Écran |
|---|---|---|
| `/connexion` | Public | Saisie du numéro de téléphone |
| `/connexion/code` | Public | Saisie du code OTP à 6 chiffres |

| Route | Accès | Écran |
|---|---|---|
| `/inscription` | Après premier OTP | Nom, prénom, photo facultative |
| `/rejoindre/:code` | Connecté | Aperçu du groupe et demande d'adhésion |
| `/` | Connecté | Accueil participant |
| `/groupes` | Connecté | Mes groupes |
| `/groupes/:id` | Membre du groupe | Détail du groupe (onglets Tour, Calendrier, Membres, Historique) |
| `/groupes/:id/payer` | Membre du groupe | Récapitulatif avant paiement |
| `/paiement/succes` et `/paiement/echec` | Connecté | Retour depuis l'agrégateur |
| `/notifications` | Connecté | Centre de notifications |
| `/profil` | Connecté | Profil, numéro de reversement, préférences de notification, déconnexion |
| `/admin` | admin, super_admin | Tableau de bord administrateur |
| `/admin/groupes/nouveau` | admin, super_admin | Assistant de création de groupe |
| `/admin/groupes/:id` | Owner ou co-admin du groupe | Gestion du groupe (onglets Aperçu, Membres, Paiements, Tours, Réglages) |
| `/admin/administrateurs` | super_admin | Liste et gestion des comptes administrateurs |
| `*` | Public | Page 404 |

Crée deux composants de garde : `RequireAuth` (redirige vers `/connexion`) et `RequireRole` (accepte une liste de rôles globaux, ou un rôle dans le groupe de l'URL ; sinon affiche une page « Accès refusé »).

## 5. Écrans participant

Chaque écran doit répondre en un coup d'œil à trois questions : combien je dois, avant quand, et quand je reçois.

**Connexion (`/connexion`, `/connexion/code`).**

- Champ téléphone avec indicatif +221 prérempli, masque `77 123 45 67`, validation 9 chiffres commençant par 7.
- Bouton « Recevoir le code », puis écran de 6 cases pour le code avec collage automatique, compte à rebours de 60 s avant « Renvoyer le code ».
- En mode simulé, le code `123456` est toujours accepté.
- Première connexion : redirection vers `/inscription` (prénom, nom, photo facultative).

**Accueil ( `/` ).**

- En-tête : « Bonjour, [prénom] » et cloche avec le nombre de notifications non lues.
- Carte résumé en vert clair : total à payer ce mois-ci, tous groupes confondus, et prochaine date limite.
- Liste « À payer » : une carte par groupe avec nom, montant, date limite, badge de statut et bouton « Payer ».
- Carte « Mon prochain tour » : groupe, date prévue et montant de la cagnotte que je recevrai.
- Bouton « Rejoindre un groupe » (saisie d'un code d'invitation).
- État vide : illustration simple et texte « Vous n'avez pas encore de groupe » avec le bouton Rejoindre.

**Mes groupes ( `/groupes` ).** Liste de cartes : nom, montant et fréquence, progression du cycle (« Tour 3 / 10 » avec barre), statut de mon paiement du tour en cours, rôle si je suis admin du groupe.

**Détail d'un groupe ( `/groupes/:id` ).** En-tête avec nom, montant, fréquence et cagnotte par tour, puis quatre onglets :

- **Tour en cours** : numéro du tour, bénéficiaire, date limite, barre « 7 / 10 ont payé », mon statut et bouton « Payer [montant] » si je n'ai pas payé.
- **Calendrier** : liste verticale de tous les tours avec date, bénéficiaire et statut (passé, en cours, à venir) ; mon tour mis en évidence en bleu clair.
- **Membres** : avatar, nom, badge de paiement du tour en cours, mention Admin pour les admins. Les numéros de téléphone ne sont pas affichés aux participants.
- **Historique** : mes paiements (date, montant, moyen, référence) et la cagnotte reçue, le cas échéant.

**Payer ( `/groupes/:id/payer` ).** Récapitulatif : groupe, tour, montant, frais éventuels, total. Choix du moyen (Wave, Orange Money, Free Money) sous forme de grandes cartes avec logo. Bouton « Payer [total] » qui lance la redirection (section 9).

**Retour de paiement ( `/paiement/succes` , `/paiement/echec` ).** Succès : coche verte, montant, référence, bouton « Retour au groupe » ; afficher « Confirmation en cours » tant que le statut n'est pas passé à « payé ». Échec : message clair et bouton « Réessayer ».

**Rejoindre ( `/rejoindre/:code` ).** Aperçu du groupe (nom, montant, fréquence, places restantes, administrateurs), bouton « Demander à rejoindre », puis message « Demande envoyée, en attente de l'administrateur ».

**Notifications ( `/notifications` ).** Liste chronologique, non lues en gras avec point bleu, tap pour ouvrir l'écran concerné, bouton « Tout marquer comme lu ».

**Profil ( `/profil` ).** Photo, nom, téléphone ; numéro de reversement (Wave ou Orange Money) où recevoir la cagnotte ; interrupteurs de notifications (rappels, bénéficiaire du mois, paiements) ; bouton « Installer l'application » si non installée ; déconnexion.

## 6. Écrans administrateur et super administrateur

L'administrateur ne voit que les groupes dont il est propriétaire ou co-admin ; le super administrateur voit tout.

**Tableau de bord ( `/admin` ).**

- Quatre indicateurs en cartes : groupes actifs, total collecté ce mois (FCFA), taux de paiement du tour en cours (%), membres en retard.
- Liste « À traiter » : demandes d'adhésion en attente, tours prêts pour le reversement, paiements échoués.
- Liste de mes groupes avec progression et bouton « Créer un groupe ».

**Création de groupe ( `/admin/groupes/nouveau` ).** Assistant en 4 étapes avec barre de progression, boutons Précédent / Suivant, validation Zod à chaque étape :

1. Informations : nom (3 à 50 caractères), description facultative.
2. Règles : montant de cotisation en FCFA (entier, minimum 500), fréquence (hebdomadaire, quinzaine, mensuelle), nombre de participants (2 à 50), jour limite de paiement, date de démarrage, pénalité de retard facultative. Afficher en direct : « Cagnotte par tour : 45 000 FCFA – Durée : 10 mois ».
3. Attribution : tirage au sort ou ordre manuel.
4. Récapitulatif : tout relire puis « Créer le groupe ». Ensuite, afficher le code et le lien d'invitation avec boutons Copier et Partager (Web Share API, repli WhatsApp).

**Gestion d'un groupe ( `/admin/groupes/:id` ).** Cinq onglets :

- **Aperçu** : statut du groupe (brouillon, en cours, terminé), tour en cours, total collecté, prochain bénéficiaire, bouton « Démarrer le cycle » (actif seulement quand le groupe est complet et l'ordre fixé).
- **Membres** : liste avec téléphone visible, demandes en attente (Accepter / Refuser), ajout par numéro, retrait d'un membre (impossible après le démarrage), section « Administrateurs du groupe » pour ajouter ou retirer un co-admin parmi les membres.

- **Paiements** : tableau du tour sélectionné (membre, statut, moyen, date, référence), filtre par statut, bouton « Enregistrer un paiement en espèces », bouton « Relancer les retardataires ».
- **Tours** : liste des tours ; lancement du tirage au sort avec animation courte et résultat visible de tous, ou ordre manuel par glisser-déposer avant le démarrage ; bouton « Reverser la cagnotte » sur le tour en cours quand tous ont payé, avec fenêtre de confirmation affichant bénéficiaire, numéro de reversement et montant.
- **Réglages** : modifier nom et description ; montant, fréquence et nombre de membres verrouillés après le démarrage (champs grisés avec explication) ; régénérer le code d'invitation ; supprimer le groupe (propriétaire seulement, brouillon seulement, confirmation en tapant le nom).

**Administrateurs ( `/admin/administrateurs` , super admin uniquement).** Liste des comptes admin (nom, téléphone, nombre de groupes, statut actif ou suspendu), bouton « Ajouter un administrateur » par numéro de téléphone, actions Suspendre / Réactiver / Retirer le rôle, avec confirmation. Statistiques globales en haut : nombre d'utilisateurs, de groupes, montant total collecté.

Toute action irréversible (reversement, retrait, suppression, suspension) passe par une fenêtre de confirmation.

## 7. Structure du projet et composants

Le code est rangé par fonctionnalité ; les composants d'interface génériques vivent dans `components/ui` et ne connaissent rien du métier.

```
src/
  app/            # App.tsx, router.tsx, providers (QueryClient, Toaster)
  components/
    ui/           # Button, Input, PhoneInput, OtpInput, Select, Card, Badge,
Tabs,
                  # Modal, ConfirmDialog, Avatar, ProgressBar, Skeleton,
EmptyState, Stepper
    layout/       # AppShell, BottomNav, SideNav, PageHeader, OfflineBanner
  features/
    auth/         # pages connexion, code, inscription + RequireAuth,
RequireRole
    groups/       # pages participant : liste, détail, rejoindre
    payments/     # page payer, retours succès / échec, PaymentMethodPicker
    notifications/
    profile/
    admin/        # dashboard, création de groupe, gestion de groupe,
administrateurs
  services/       # interfaces + implémentations mock (section 8)
```

```
  mocks/          # données de démonstration
  hooks/          # useAuth, useOnline, useInstallPrompt, usePushPermission
  lib/            # format.ts (FCFA, dates), validators.ts (schémas Zod)
  types/          # types métier (section 8)
  styles/         # index.css (Tailwind)
public/
  icons/          # icônes PWA 192, 512, maskable
```

**Composants métier à créer.**

| Composant | Rôle |
|---|---|
| `AmountText` | Affiche un montant formaté « 45 000 FCFA » |
| `StatusBadge` | Badge payé, en attente, en retard, ton tour, échoué |
| `GroupCard` | Carte de groupe pour les listes |
| `RoundProgress` | Barre « 7 / 10 ont payé » |
| `CycleProgress` | Barre « Tour 3 / 10 » |
| `RoundTimeline` | Calendrier vertical des tours |
| `MemberRow` | Ligne membre avec avatar, nom, badges |
| `PayButton` | Bouton vert « Payer [montant] » |
| `PaymentMethodPicker` | Choix Wave, Orange Money, Free Money |
| `StatCard` | Indicateur du tableau de bord admin |
| `InviteShare` | Code, lien, boutons Copier et Partager |
| `DrawAnimation` | Animation courte du tirage au sort |

Chaque page gère trois états : chargement ( `Skeleton` ), vide ( `EmptyState` ) et erreur (message + bouton Réessayer).

## 8. Types et couche de données simulée

Les pages n'appellent jamais les données directement : elles passent par des hooks TanStack Query qui appellent des services, eux-mêmes implémentés en mock aujourd'hui et en Supabase plus tard.

```
// src/types/index.ts
export type GlobalRole = 'super_admin' | 'admin' | 'user';
export type GroupRole = 'owner' | 'co_admin' | 'member';
export type Frequency = 'weekly' | 'biweekly' | 'monthly';
export type GroupStatus = 'draft' | 'active' | 'finished';
export type AttributionMode = 'random' | 'manual';
export type MembershipStatus = 'pending' | 'active' | 'removed';
export type RoundStatus = 'upcoming' | 'open' | 'ready_for_payout' |
'paid_out';
export type PaymentStatus = 'pending' | 'paid' | 'late' | 'failed' |
'refunded';
export type PaymentMethod = 'wave' | 'orange_money' | 'free_money' | 'cash';

export interface User {
  id: string; firstName: string; lastName: string; phone: string;
  avatarUrl?: string; globalRole: GlobalRole; isSuspended: boolean;
  payoutPhone?: string; payoutMethod?: 'wave' | 'orange_money';
}

export interface Group {
  id: string; name: string; description?: string;
  contributionAmount: number;        // FCFA, entier
  frequency: Frequency; maxMembers: number;
  paymentDueDay: number; startDate: string; // ISO
  attributionMode: AttributionMode; latePenalty?: number;
  status: GroupStatus; inviteCode: string; createdBy: string;
}

export interface Membership {
  id: string; groupId: string; userId: string;
  role: GroupRole; status: MembershipStatus; payoutOrder?: number; joinedAt:
string;
}

export interface Round {
  id: string; groupId: string; number: number;
  openDate: string; dueDate: string; beneficiaryId: string;
  potAmount: number; status: RoundStatus; paidOutAt?: string;
}

export interface Payment {
  id: string; roundId: string; userId: string; amount: number;
  method: PaymentMethod; providerRef?: string; status: PaymentStatus;
paidAt?: string;
}
```

```
export interface AppNotification {
  id: string; userId: string; type: string; title: string;
  body: string; link?: string; read: boolean; createdAt: string;
}
```

**Services à définir** (une interface par fichier, plus une implémentation `mock`) : `authService` (sendOtp, verifyOtp, completeProfile, logout), `groupService` (listMine, get, create, update, start, getByInviteCode, requestJoin), `membershipService` (list, accept, reject, remove, setRole), `roundService` (list, drawOrder, setManualOrder, payout), `paymentService` (listForRound, initiatePayment, recordCash, remindLate), `notificationService` (list, markRead, markAllRead), `adminService` (stats, listAdmins, addAdmin, suspend, reactivate, revokeRole).

**Données de démonstration** (`src/mocks/`) :

- 1 super admin, 2 administrateurs, 12 participants avec noms sénégalais réalistes et numéros fictifs en 77, 78, 76, 70.
- 3 groupes : « Tontine Famille » (4 500 FCFA, mensuelle, 10 membres, tour 3 en cours), « Collègues du bureau » (10 000 FCFA, mensuelle, 6 membres, en brouillon), « Amies du quartier » (2 000 FCFA, hebdomadaire, 8 membres, terminée).
- Un mélange de paiements payés, en attente et en retard sur le tour en cours, et une dizaine de notifications.
- Les services mock ajoutent un délai de 300 à 600 ms et gardent l'état en mémoire pendant la session.
- Un sélecteur de compte de démonstration sur l'écran de connexion (visible seulement si `VITE_USE_MOCKS=true`) permet de se connecter en super admin, admin ou participant.

## 9. Paiement, PWA et notifications

Côté front, le paiement se résume à une redirection puis à l'attente d'une confirmation ; le front ne décide jamais seul qu'un paiement est réussi.

**Paiement (agrégateur PayDunya ou PayTech, branché plus tard côté backend).**

1. `paymentService.initiatePayment({ roundId, method })` renvoie `{ paymentId, checkoutUrl }`.
2. Le front redirige vers `checkoutUrl` avec `window.location.assign`.
3. L'agrégateur renvoie l'utilisateur sur `/paiement/succes?payment=<id>` ou `/paiement/echec?payment=<id>`.

4. La page de succès interroge le statut toutes les 3 secondes (maximum 60 secondes) jusqu'à `paid` ; sinon elle affiche « Paiement en cours de vérification, vous serez notifié ».

En mode mock, `checkoutUrl` pointe vers une page interne `/paiement/simulation` avec deux boutons « Simuler un succès » et « Simuler un échec ».

**PWA.**

- vite-plugin-pwa en mode `generateSW`, `registerType: 'prompt'` : quand une nouvelle version est disponible, afficher un bandeau « Nouvelle version disponible – Mettre à jour ».
- Manifeste : nom « Tontine », nom court « Tontine », `display: standalone`, `orientation: portrait`, `lang: fr`, couleurs de la section 3, icônes 192, 512 et maskable.
- Cache : fichiers de l'application en précache ; réponses de données en `NetworkFirst` pour pouvoir consulter groupes, calendrier et historique hors ligne.
- Hors ligne : bandeau `OfflineBanner` en haut, boutons de paiement et actions admin désactivés avec le message « Connexion requise ».
- Installation : hook `useInstallPrompt` qui capte `beforeinstallprompt` (Android) et affiche un bouton « Installer l'application » ; sur iPhone, afficher à la place une aide « Partager puis Sur l'écran d'accueil ».

**Notifications push.**

- Ne jamais demander la permission au chargement : afficher d'abord un encart expliquant l'intérêt (« Recevez les rappels de cotisation »), puis demander au clic.
- Hook `usePushPermission` : état de la permission, abonnement via `PushManager.subscribe` avec `VITE_VAPID_PUBLIC_KEY`, envoi de l'abonnement à `notificationService.savePushSubscription` (mock pour l'instant).
- Service worker : afficher la notification reçue et, au clic, ouvrir le `link` de la notification.
- Sur iPhone, le push ne fonctionne que si l'application est installée sur l'écran d'accueil : l'expliquer dans l'encart.

## 10. Règles UX et qualité

L'application doit être simple pour des utilisateurs peu habitués aux applications, sur des téléphones modestes et une connexion 3G.

**Affichage.**

- Montants : `Intl.NumberFormat('fr-FR')` sans décimales, suivi de « FCFA » (exemple : 45 000 FCFA), centralisé dans `lib/format.ts`.
- Dates : format court « 5 nov. 2026 » et relatif quand utile (« dans 3 jours »), via date-fns en français.
- Téléphones affichés « +221 77 123 45 67 ».
- Textes courts, verbes d'action sur les boutons (« Payer 4 500 FCFA » plutôt que « Valider »).

**Mobile d'abord.** Conception à 360 px de large, puis adaptation tablette et ordinateur. Zones tactiles d'au moins 44 × 44 px. Action principale en bas d'écran, à portée de pouce. Respect des zones sûres ( `env(safe-area-inset-*)` ) en mode installé.

**Retour utilisateur.** Toast de confirmation après chaque action réussie, message d'erreur compréhensible sans jargon, boutons désactivés avec indicateur pendant les envois pour éviter les doubles paiements.

**Accessibilité.** Contraste AA minimum, libellés sur tous les champs, `aria-label` sur les boutons icônes, navigation clavier possible, focus visible en bleu. Le statut ne repose jamais sur la couleur seule : badge avec texte et icône.

**Performance.** Pages chargées à la demande ( `React.lazy` par route), images d'avatar compressées, score Lighthouse PWA et Performance ⩾ 90 sur mobile.

**Qualité du code.** TypeScript strict, ESLint sans erreur, composants de moins de 200 lignes, schémas Zod partagés dans `lib/validators.ts` , tests Vitest pour `format.ts` , les validateurs, `RequireRole` et le calcul de la cagnotte.

## 11. Ordre de travail et critères d'acceptation

Travaille dans cet ordre et arrête-toi à la fin de chaque étape pour que je puisse tester avant la suivante.

1. **Socle** : projet Vite + TypeScript, Tailwind avec les couleurs de la section 3, police Inter, ESLint et Prettier, structure de dossiers de la section 7.
2. **Composants UI** : tous les composants de `components/ui` , plus une page `/demo-ui` (mode mock seulement) qui les affiche tous.
3. **Types, mocks et services** : section 8 complète, avec le sélecteur de compte de démonstration.
4. **Authentification et routes** : connexion, code OTP, inscription, `RequireAuth` , `RequireRole` , `AppShell` avec navigation du bas et menu latéral.
5. **Parcours participant** : accueil, mes groupes, détail du groupe et ses 4 onglets, rejoindre un groupe, notifications, profil.

6. **Paiement** : page Payer, simulation, pages succès et échec avec vérification du statut.
7. **Parcours administrateur** : tableau de bord, assistant de création, gestion d'un groupe et ses 5 onglets, co-admins, tirage au sort, reversement.
8. **Super administrateur** : page Administrateurs et statistiques globales.
9. **PWA** : manifeste, icônes, service worker, mode hors ligne, invite d'installation, mise à jour, permission et réception des push.
10. **Finitions** : états vides, chargement et erreur partout, accessibilité, tests Vitest, audit Lighthouse.

**Critères d'acceptation.**

- [ ] `npm run dev`, `npm run build` et `npm run test` passent sans erreur ni avertissement TypeScript.
- [ ] Connecté en participant, je vois mes montants dus, je paie un tour en simulation et le statut passe à « payé » partout.
- [ ] Connecté en administrateur, je crée un groupe de 4 500 FCFA à 10 membres et l'écran annonce 45 000 FCFA par tour sur 10 mois.
- [ ] Un administrateur ne voit pas les groupes des autres ; un co-admin gère le groupe mais ne peut pas le supprimer.
- [ ] Seul le super administrateur accède à `/admin/administrateurs` ; les autres voient « Accès refusé ».
- [ ] Les champs montant, fréquence et nombre de membres sont verrouillés après le démarrage du cycle.
- [ ] L'application s'installe sur Android, s'ouvre en plein écran et affiche les groupes déjà consultés sans connexion.
- [ ] Seules les couleurs de la charte apparaissent dans l'interface.
- [ ] Tout le texte visible est en français.

**Hors périmètre de ce travail.** Backend Supabase, vraie intégration PayDunya ou PayTech, envoi réel de SMS et de notifications push depuis un serveur, mode sombre, traduction en wolof.